

2. SQL advanced

Content

2.1. Subqueries

2.2. DML

2.3. Views

2.4. Common table expressions (CTEs)

2.5. Window Functions

2.1. Subqueries

Basic form

- Nested subqueries
 - Basic form

```
SELECT  
FROM  
WHERE condition
```



Contains in its left- and/or right-hand side statement another SELECT

- Outer level query = first SELECT = the main question
- Inner level query = SELECT in WHERE/HAVING = the subquery
 - Always executed first
 - Always between (and)
 - Subqueries can be nested at > 1 level
- A subquery can return **one value** or **a list of values**

Returning a single value

- Result of subquery = used anywhere in an expression
 - With all relational operators: =, >, <, <=, >=, <>
 - Example:
 - What is the UnitPrice of the most expensive product?

```
SELECT MAX(UnitPrice) As MaxPrice FROM Products
```

	MaxPrice
1	263,50

- What is the most expensive product?

```
SELECT ProductID, ProductName, UnitPrice As MaxPrice  
FROM Products  
WHERE UnitPrice = (SELECT MAX(UnitPrice) FROM Products)
```

	ProductID	ProductName	MaxPrice
1	38	Côte de Blaye	263,50

- What is the difference with the following?

```
SELECT TOP 1 ProductID, ProductName, UnitPrice As MaxPrice  
FROM Products ORDER BY UnitPrice DESC
```

Returning a single value

- Other examples
 - Give the products that cost more than average

```
SELECT ProductID, ProductName, UnitPrice As MaxPrice
FROM Products
WHERE UnitPrice > (SELECT AVG(UnitPrice) FROM Products)
```

- Who is the youngest employee from the USA?

```
SELECT LastName, FirstName
FROM Employees
WHERE Country = 'USA'
AND BirthDate = (SELECT MAX(BirthDate) FROM Employees WHERE Country = 'USA')
```

Returning a single column

- The resulting column can be used as a list
 - Operators IN, NOT IN, ANY, ALL
 - IN operator (=ANY operator)
 - Example: Give all employees that have processed orders

```
SELECT e.EmployeeID, e.FirstName + ' ' + e.LastName As Name
FROM Employees e
WHERE e.EmployeeID IN (SELECT DISTINCT EmployeeID FROM Orders)
```

- This can also be accomplished with JOIN

```
SELECT DISTINCT e.EmployeeID, e.FirstName + ' ' + e.LastName As Name
FROM Employees e JOIN Orders o ON e.EmployeeID = o.EmployeeID
```

Returning a single column

- The resulting column can be used as a list
 - Operators IN, NOT IN, ANY, ALL
 - NOT IN operator
 - Example: Give all customers that have not placed any orders yet

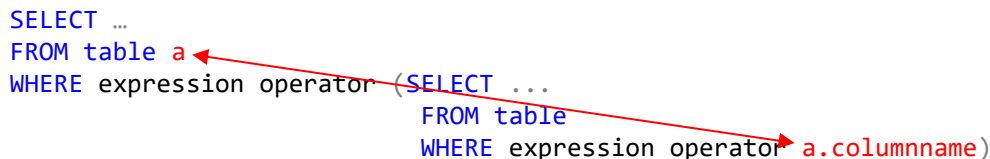
```
SELECT *  
FROM Customers  
WHERE CustomerID NOT IN (SELECT DISTINCT CustomerID FROM Orders)
```

- This can also be accomplished with JOIN

```
SELECT *  
FROM Customers c LEFT JOIN Orders o ON c.CustomerID = o.CustomerID  
WHERE o.CustomerID is NULL
```

Correlated subqueries

- Principle



```
SELECT ...  
FROM table a  
WHERE expression operator (SELECT ...  
                           FROM table  
                           WHERE expression operator a.columnname)
```

- Remark: in the inner query you can use fields from the tables in the outer query but NOT vice versa
- For performance reasons use joins or CTE (see further) or simple subquery if possible

Correlated subqueries

- When the inner query depends on information from the outer query
 - Subquery contains condition referring to main query
- Subquery is executed for each row in main query
 - Order of execution is from top to bottom → **$O(n^2)$!**
 - Not from bottom to top as in simple subqueries → $O(n)$

Correlated subqueries

1. Next row in the outer query

2. Outer query passes column values
for that row to inner query

3. Inner query uses those values
to evaluate inner query

4. Inner query returns value to outer query,
which decides if row in outer query will be kept



Back to step 1.

Correlated subqueries

- Example: Employees with salary larger than the average

```
SELECT FirstName + ' ' + LastName As FullName, Salary
FROM Employees
WHERE Salary > (SELECT AVG(Salary) FROM Employees)
```

- Example: Employees with salary larger than the average for those reporting to the same boss

```
SELECT FirstName + ' ' + LastName As FullName, ReportsTo, Salary
FROM Employees As e
WHERE Salary > (SELECT AVG(Salary) FROM Employees WHERE ReportsTo = e.ReportsTo)
```

Completely
different!

Subqueries and the EXISTS operator

- The operator EXISTS tests the existence of a result set
 - Example: All customers that already placed an order

```
SELECT *  
FROM Customers As c  
WHERE EXISTS  
(SELECT * FROM Orders WHERE CustomerID = c.customerID)
```

- There is also NOT EXISTS
 - Example: All customers that have not placed any orders yet

```
SELECT *  
FROM Customers As c  
WHERE NOT EXISTS  
(SELECT * FROM Orders WHERE CustomerID = c.customerID)
```

3 ways to accomplish the same result

- Example: All customers that did not place any orders yet
 - OUTER JOIN

```
SELECT *  
FROM Customers c LEFT JOIN Orders o ON c.CustomerID = o.CustomerID  
WHERE o.CustomerID is NULL
```

- Simple subquery

```
SELECT *  
FROM Customers  
WHERE CustomerID NOT IN (SELECT DISTINCT CustomerID FROM Orders)
```

- Correlated subquery

```
SELECT *  
FROM Customers As c  
WHERE NOT EXISTS  
(SELECT * FROM Orders WHERE CustomerID = c.customerID)
```

Which one will be the slowest?

Subqueries in **SELECT** and **FROM**

- The previous examples showed how subqueries can be used in the WHERE-clause
- Since the result of a query is a table, it can also be used in the FROM-clause
- We will be using CTE's (see further) instead

Subqueries in the SELECT-clause

- Subqueries can be used in SELECT (simple or correlated)
 - Example: Give for each employee how much they earn more (or less) than the average salary of all employees with the same supervisor
 - Only for illustration, we will be using CTEs instead

```
SELECT Lastname, Firstname, Salary,  
Salary -  
(  
    SELECT AVG(Salary)  
    FROM Employees  
    WHERE ReportsTo = e.ReportsTo  
)  
FROM Employees e
```

Exercises

- 1. Give the id and name of the products that have not been purchased yet.
- 2. Select the names of the suppliers who supply products that have not been ordered yet.
- 3. Give a list of all customers from the same country as the customer Maison Dewey
- 4. Give for each product how much the price differs from the average price of all products of the same category
- 5. Give per title the employee that was last hired
- 6. Which employee has processed most orders?
- 7. What's the most common ContactTitle in Customers?
- 8. Is there a supplier that has the same name as a customer?
- 9. Give all the orders for which the ShipAddress is different from the CustomerAddress,

2.2. DML

SQL - DML basic tasks

- SELECT → consulting data
- INSERT → adding data
- UPDATE → changing data
- DELETE → removing data
- MERGE → combine INSERT, UPDATE and DELETE

Tip for not destroying your database

- Caution!
 - The statements in this chapter are destructive
 - SQL has no UNDO by default!
 - BUT you can 'simulate' UNDO if you take precautions

```
/* Tip for not destroying your database */
BEGIN TRANSACTION -- starts a new "transaction" -> Saves previous state of DB in buffer

-- several "destructive" commands can go here:
INSERT INTO Products(ProductName)
values ('TestProduct');

-- only you (in your session) can see changes
SELECT * FROM Products WHERE ProductID = (SELECT MAX(ProductID) FROM Products)

ROLLBACK; --> ends transaction and restores database in previous state
-- COMMIT; --> ends transaction and makes changes permanent
```

Transactions?
See next
chapters!

INSERT - Adding data

- The INSERT statement adds data in a table
 - Add one row through via specification
 - Add selected row(s) from other tables

INSERT of 1 row

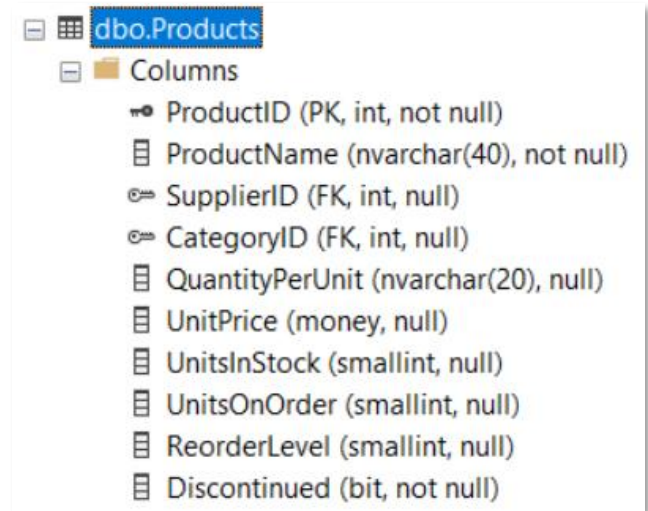
- Method 1: specify only the (not NULL) values for specific columns
- Method 2: specify all column values
- Auto-numbered fields cannot be mentioned

```
INSERT INTO Products (ProductName, CategoryID, Discontinued)
VALUES ('Toblerone', 3, 0)
```

```
INSERT INTO Products
VALUES ('Sultana', null, 3, null, null, null, null, null, 1)
```

INSERT of 1 row - requirements

- # specified columns = # values
- Specified values and columns have compatible data types
- No column names specified → order of CREATE TABLE
- Unmentioned columns get
 - DEFAULT value if specified
 - NULL value otherwise
- NULL can be specified as a value



The screenshot shows the 'dbo.Products' table structure in SQL Server Enterprise Manager. The table has the following columns:

Column Name	Data Type	Constraints
ProductID	int	PK, not null
ProductName	nvarchar(40)	not null
SupplierID	int	FK, null
CategoryID	int	FK, null
QuantityPerUnit	nvarchar(20)	null
UnitPrice	money	null
UnitsInStock	smallint	null
UnitsOnOrder	smallint	null
ReorderLevel	smallint	null
Discontinued	bit	not null

INSERT of rows from other tables

- Mandatory fields have to be specified, unless they have a DEFAULT value
- Constraints (see further) are validated
- Unmentioned columns get the value NULL or the DEFAULT value if any

```
INSERT INTO Customers (CustomerID, ContactName, ContactTitle, CompanyName)
SELECT substring(FirstName,1,2) + substring(LastName,1,3), FirstName + ' ' +
LastName, Title, 'EmployeeCompany'
FROM Employees
```

UPDATE - Changing data

- Changing all rows in a table
 - Example: Increase the price of all products with 10%

```
UPDATE Products  
SET UnitPrice = UnitPrice * 1.1
```

- Changing 1 row or a group of rows
 - Example: Increase price of all 'Bröd' products with 10%

```
UPDATE Products  
SET UnitPrice = UnitPrice * 1.1  
WHERE ProductName LIKE '%Bröd%'
```

- Example: Increase price of all 'Bröd' products with 10% and set all units in stock to 0

```
UPDATE Products  
SET UnitPrice = UnitPrice * 1.1, UnitsInStock = 0  
WHERE ProductName LIKE '%Bröd%'
```

UPDATE - Changing data

- Change rows based on data in another table
 - Standard SQL does not offer JOINS in an update statement → you can only use subqueries to refer to another table
 - Example: Due to a change in the euro – dollar exchange rate, we have to increase the unit price of products delivered by suppliers from the USA by 10%

```
UPDATE Products
SET UnitPrice = (UnitPrice * 1.1)
WHERE SupplierID IN
(SELECT SupplierID FROM Suppliers WHERE Country = 'USA')
```

subquery

DELETE - Removing data

- Delete some rows
 - Example: Delete the 'Bröd' products

```
DELETE FROM Products
WHERE ProductName LIKE '%Bröd%'
```

- Delete all rows in a table
 - via DELETE – the ID value continues
 - via TRUNCATE – the ID value restarts from 1
 - TRUNCATE is also more performant
 - but does not offer WHERE clause: it's all or nothing

```
-- the identity value continues
DELETE FROM Products
-- the identity value restarts from 1
TRUNCATE TABLE Products
```

DELETE - based on another table

- Change rows based on data in another table
 - Again, no JOIN, only subquery
 - Example: Delete the order details for all orders from the most recent order date

```
DELETE FROM OrderDetails
WHERE OrderID IN
(SELECT OrderID FROM Orders WHERE OrderDate = (SELECT MAX(OrderDate) from Orders))
```

MERGE

- MERGE combines INSERT, UPDATE and DELETE
- Very common use case
 - Users work on an Excel sheet to update a relatively large number of records because Excel offers a better overview than their ERP tool
 - They can update records, add new ones and delete records in Excel
 - After uploading the edited Excel file to a temporary table, the merge statement performs all UPDATES, INSERTs and DELETES at once

MERGE

- Script to create temporary table

```
/* First execute following script to simulate the Excel file that has been imported  
to a temporary table ShippersUpdate */
```

```
DROP TABLE IF EXISTS ShippersUpdate;  
-- Add everything from Shippers to ShippersUpdate  
SELECT * INTO ShippersUpdate FROM Shippers  
  
-- Add an extra record to ShippersUpdate  
INSERT INTO ShippersUpdate VALUES ('Pickup', '(503) 555-9647')  
  
-- Update a record of ShippersUpdate  
UPDATE ShippersUpdate SET Phone = '(503) 555-4512' WHERE ShipperID = 1  
  
-- Remove a record from ShippersUpdate  
DELETE FROM ShippersUpdate WHERE shipperID = 4
```

MERGE

Original table Shippers

	ShipperID	CompanyName	Phone
1	1	Speedy Express	(503) 555-9831
2	2	United Package	(503) 555-3199
3	3	Federal Shipping	(503) 555-9931
4	4	Total Shipping	(503) 555-9752
5	5	Federal Express	(503) 555-9773
6	7	PostNL	(503) 555-1236

Temporary table ShippersUpdate

	ShipperID	CompanyName	Phone
1	1	Speedy Express	(503) 555-4512
2	2	United Package	(503) 555-3199
3	3	Federal Shipping	(503) 555-9931
4	5	Federal Express	(503) 555-9773
5	6	Pickup	(503) 555-9647

delete

insert

update

Many
differences

MERGE

```
BEGIN TRANSACTION

INSERT INTO Shippers
VALUES ('PostNL', '(503) 555-1236')

SELECT * FROM Shippers;
SELECT * FROM ShippersUpdate;
```

MERGE

```
MERGE Shippers as t -- t = target
USING ShippersUpdate as s -- s = source
ON (t.ShipperID = s.ShipperID)
```

```
-- Which rows are in source and have different values for CompanyName or Phone?
-- Update those rows in target with the values coming from source
WHEN MATCHED AND t.CompanyName <> s.CompanyName OR ISNULL(t.Phone, '') <> ISNULL(s.Phone, '')
THEN UPDATE SET t.CompanyName = s.CompanyName, t.Phone=s.Phone
```

```
-- Which rows are in target and not in source?
-- Add those rows to source
WHEN NOT MATCHED BY target --> new rows
THEN INSERT (CompanyName, Phone) VALUES (s.CompanyName, s.Phone)
```

```
-- Which rows are in source and not in target?
-- Delete those rows from target
WHEN NOT MATCHED BY source --> rows to delete
THEN DELETE;
```

```
-- Check the result
SELECT * FROM Shippers
ROLLBACK;
```

	ShipperID	CompanyName	Phone
1	1	Speedy Express	(503) 555-4512
2	2	United Package	(503) 555-3199
3	3	Federal Shipping	(503) 555-9931
4	5	Federal Express	(503) 555-9773
5	8	Pickup	(503) 555-9647

REMARK
The option to
delete records is
a non-standard
extension of
MS SQL Server

2.3. Views

Introduction

- A **view** is a saved SELECT statement
 - It can be seen as a virtual table composed of other tables & views
 - No data is stored in the view itself, at each referral the underlying SELECT is re-executed

```
CREATE VIEW view_name [(column_list)]AS select_statement
```

Advantages

- Hide complexity of the database
 - Hide complex database design
 - Make large and complex queries accessible and reusable
 - Can be used as a partial solution for complex problems
- Used for securing data access
 - Revoke access to tables
 - Grant access to customized views
- Organize data for export to other applications

Requirements

- # columns in (column_list) = # columns in select
 - No column names specified? → taken from the select
 - Column names are mandatory if select statement contains calculations or joins in which some column names appear more than once
- Select statement may not contain an order by

```
CREATE VIEW view_name [(column_list)]AS select_statement
```

Views – CRUD operations

-- Creating a view

```
CREATE VIEW V_ProductsCustomer(productcode, company, quantity)
AS SELECT od.ProductID, c.CompanyName, sum(od.Quantity)
FROM Customers c
JOIN Orders o ON o.CustomerID = c.CustomerID
JOIN OrderDetails od ON o.OrderID = od.OrderID
GROUP BY od.ProductID, c.CompanyName;
```

-- Using a view

```
SELECT * FROM V_ProductsCustomer;
```

-- Changing a view

```
ALTER VIEW V_ProductsCustomer(productcode, company)
AS SELECT od.ProductID, c.CompanyName
FROM Customers c
JOIN Orders o ON o.CustomerID = c.CustomerID
JOIN OrderDetails od ON o.OrderID = od.OrderID
GROUP BY od.ProductID, c.CompanyName;
```

-- Dropping a view

```
DROP VIEW V_ProductsCustomer;
```

Example

- Example: Create a view with the number of orders per employee per year
- Example: Calculate per employee and per year the running total of processed orders

```
-- Create a view with the number of orders per employee per year
CREATE OR ALTER VIEW vw_number_of_orders_per_employee_per_year
AS
SELECT EmployeeID, YEAR(OrderDate) As OrderYear, COUNT(OrderID) As NumberOfOrders
FROM Orders
GROUP BY EmployeeID, YEAR(OrderDate)

-- Check the result
SELECT * FROM vw_number_of_orders_per_employee_per_year
```

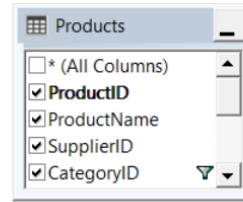
Example

```
-- Add the running total. Use the view.  
SELECT EmployeeID, OrderYear, NumberOfOrders,  
  (SELECT SUM(NumberOfOrders)  
   FROM vw_number_of_orders_per_employee_per_year  
   WHERE OrderYear <= vw.OrderYear AND EmployeeID = vw.EmployeeID  
  ) As TotalNumberOfOrders  
FROM vw_number_of_orders_per_employee_per_year vw  
ORDER BY EmployeeID, OrderYear
```

- Drawback: views are stored in the database
 - Might create a mess if you have many
 - Solution = CTEs (see further)

SQL Server Management Studio

- Simple views via the UI
- Not possible for views with subqueries or common table expressions



	Column	Alias	Table	Output	Sort Type	Sort Order	Filter
▶	ProductID		Products	☒			
	ProductName		Products	☒			
	SupplierID		Products	☒			
	CategoryID		Products	☒			= 1
	QuantityPerUnit		Products	☒			
	UnitPrice		Products	☒			
	UnitsInStock		Products	☒			
	UnitsOnOrder		Products	☒			
	ReorderLevel		Products	☒			
	Discontinued		Products	☒			
				☐			

```
SELECT ProductID, ProductName, SupplierID, CategoryID, QuantityPerUnit,  
UnitPrice, UnitsInStock, UnitsOnOrder, ReorderLevel, Discontinued  
FROM   dbo.Products  
WHERE  (CategoryID = 1)
```

SQL Server Management Studio

- Easiest way to change existing view
 - Right clicking on the name of the view
 - Script View as...
 - ALTER to...
 - New Query Editor Window

Exercises

-- Exercise 1

- The company wants to weekly check the stock of their products.
- If the stock is below 15, they'd like to order more to fulfill the need.
- (1.1) Create a QUERY that shows the ProductId, ProductName and the name of the supplier, do not forget the WHERE clause.
- (1.2) Turn this SELECT statement into a VIEW called: vw_products_to_order.
- (1.3) Query the VIEW to see the results.

-- Exercise 2

- The company has to increase prices of certain products. To make it seem the prices are not increasing dramatically they're planning to spread the price increase over multiple years. In total they'd like a 10% price for certain products. The list of impacted products can grow over the coming years.
- We'd like to keep all the logic of selecting the correct products in 1 SQL View, in programming terms 'keeping it DRY'.
- The updating of the items is not part of the view itself.
- The products in scope are all the products with the term 'Bröd' or 'Biscuit'.
- (2.1) Create a simple SQL Query to get the correct resultset
- (2.2) Turn this SELECT statement into a VIEW called: vw_price_increasing_products.
- (2.3) Query the VIEW to see the results.



2.4. Common table expressions (CTEs)

Example

Example: Give per category the minimum price and all products with that minimum price

- Solution 1: with subquery

```
-- Solution 1 -> with subqueries
```

```
SELECT CategoryID, ProductID, UnitPrice  
FROM Products p  
WHERE UnitPrice = (SELECT MIN(UnitPrice) FROM Products WHERE CategoryID = p.CategoryID)
```



- Not performant! Loops through all products and calculates **each time** the MIN(unitprice) for its category: $O(n^2)$

The WITH component

- Solution 2: with **CTE** (Common Table Expression)

```
-- Solution 2 -> with CTE's
WITH CategoryMinPrice(CategoryID, MinPrice)
AS (SELECT CategoryID, MIN(UnitPrice)
    FROM Products p
    GROUP BY CategoryID)

SELECT c.CategoryID, p.ProductID, MinPrice
FROM Products p
JOIN CategoryMinPrice c ON p.CategoryID = c.CategoryID AND p.UnitPrice = c.MinPrice;
```



- Using WITH you can give the subquery its own name (with column names) and reuse it in the rest of the query (possibly several times!)

The WITH component

TIP

Give columns of the CTE a name!

method 1

method 2

```
-- Solution 2
WITH CategoryMinPrice(CategoryID, MinPrice)
AS (SELECT CategoryID, MIN(UnitPrice)
    FROM Products p
    GROUP BY CategoryID)
```

```
SELECT c.CategoryID, p.ProductID, MinPrice
FROM Products p
JOIN CategoryMinPrice c ON p.CategoryID = c.CategoryID AND p.UnitPrice = c.MinPrice;
```

```
-- Solution 2
WITH CategoryMinPrice
AS (SELECT CategoryID AS CategoryID, MIN(UnitPrice) AS MinPrice
    FROM Products p
    GROUP BY CategoryID)
```

```
SELECT c.CategoryID, p.ProductID, MinPrice
FROM Products p
JOIN CategoryMinPrice c ON p.CategoryID = c.CategoryID AND p.UnitPrice = c.MinPrice;
```

The WITH component

- Applications
 - Simplify SQL-instructions
 - Shorter (no repetition in SQL)
 - Faster (no repetition in execution of SQL)
 - Traverse recursively hierarchical and network structures

CTEs versus Views

- Similarities
 - WITH ~ CREATE VIEW
 - Both virtual tables (content derived from other tables)
- Differences
 - A CTE only exists during the SELECT-statement
 - A CTE is not visible for other users and applications

CTEs versus Subqueries

- Similarities
 - Both virtual tables (content derived from other tables)
- Differences
 - A CTE can be reused in the same query
 - A subquery is defined in the clause where it is used (SELECT/FROM/WHERE/...), whereas a CTE is defined on top of the query
 - A simple subquery can always be replaced by a CTE

More than 1 WITH component

- Example: Give per year per customer the relative contribution of this customer to the total revenue
- Step 1: Calculate the total revenue per year

```
-- Step 1 -> Total revenue per year

SELECT YEAR(OrderDate), SUM(od.UnitPrice * od.Quantity)
FROM Orders o INNER JOIN OrderDetails od
ON o.OrderID = od.OrderID
GROUP BY YEAR(OrderDate)
```

More than 1 WITH component

- Step 2: Calculate the total revenue per year per customer

```
-- Step 2 -> Total revenue per year per customer

SELECT YEAR(OrderDate), o.CustomerID, SUM(od.UnitPrice * od.Quantity)
FROM Orders o INNER JOIN OrderDetails od
ON o.OrderID = od.OrderID
GROUP BY YEAR(OrderDate), o.CustomerID
```

More than 1 WITH component

- Step 3: Combine both

-- Step 3 -> Combine both

```
WITH TotalRevenuePerYear(RevenueYear, TotalRevenue)
```

```
AS
```

```
(SELECT YEAR(OrderDate), SUM(od.UnitPrice * od.Quantity)
```

```
FROM Orders o INNER JOIN OrderDetails od
```

```
ON o.OrderID = od.OrderID
```

```
GROUP BY YEAR(OrderDate)),
```

```
TotalRevenuePerYearPerCustomer(RevenueYear, CustomerID, Revenue)
```

```
AS
```

```
(SELECT YEAR(OrderDate), o.CustomerID, SUM(od.UnitPrice * od.Quantity)
```

```
FROM Orders o INNER JOIN OrderDetails od
```

```
ON o.OrderID = od.OrderID
```

```
GROUP BY YEAR(OrderDate), o.CustomerID)
```

```
SELECT CustomerID, pc.RevenueYear, FORMAT(Revenue / TotalRevenue, 'P') AS Relative
```

```
FROM TotalRevenuePerYearPerCustomer pc INNER JOIN TotalRevenuePerYear t
```

```
ON pc.RevenueYear = t.RevenueYear
```

```
ORDER BY 2 ASC, 3 DESC
```

	CustomerID	RevenueYear	RelativePart
1	ERNSH	2016	7.58%
2	QUICK	2016	5.62%
3	QUEEN	2016	5.42%
4	PICCO	2016	5.38%
5	SAVEA	2016	5.38%
6	RATTC	2016	4.79%
7	FRANK	2016	4.72%
8	HUNGO	2016	4.66%
9	BLONP	2016	4.41%
10	SPLIR	2016	3.71%
11	SUPRD	2016	2.84%
12	MEREP	2016	2.70%
13	SEVES	2016	2.62%
14	LILAS	2016	2.59%
15	OLDWO	2016	2.24%

Recursive SELECTs

- **Recursivity**
 - We continue to execute a table expression until a condition is reached
 - Use the CTE in the CTE!
- Elegantly solve problems like
 - Friends of the friends of my friends (social network)
 - Hierarchy of an organization
 - Parts, subparts, subsubparts, etc. of a product (=Bill of materials)

Recursive SELECTs

- Example
 - The integers from 1 to 5
- Characteristics
 - The with component consists of (at least) 2 expressions, combined with union all (here: 1 and number + 1)
 - A temporary table is consulted in the second expression
 - At least one of the expressions may not refer to the temporary table

```
WITH numbers(number) AS  
(SELECT 1  
 UNION all  
  SELECT number + 1  
   FROM numbers  
  WHERE number < 5)  
  
SELECT * FROM numbers;
```

How does it work?

- 1. SQL searches the table expressions that don't contain recursivity and executes them one by one

```
SELECT 1
```

	number
1	1

- 2. Execute all recursive expressions. The numbers table, that got a value of 1 in step 1, is used. This row is added to the numbers table

```
SELECT number + 1  
FROM numbers  
WHERE number < 5
```

	number
1	2

How does it work?

- 3. Now the recursion starts: the 2nd expression is re-executed:

	number
1	3

Remark: only the rows added in the previous step are processed (1 row in this example)

- 4. Repeat step 3 until no new rows are added

	number
1	4

	number
1	5

How does it work?

- The last time, the expression does not return a result anymore (starting from number = 5):
`SELECT number + 1 FROM numbers WHERE number < 5`
- Summary: the 1st (non-recursive) expression is executed once, and the 2nd expression is executed until it does not return any more results

Caution

SQL Server = limited to 100 recursions by default

- Example: Give the numbers from 1 to 999

```
WITH numbers(number) AS
(SELECT 1
 UNION all
  SELECT number + 1
   FROM numbers
  WHERE number < 999)

SELECT * FROM numbers;
```

Msg 530, Level 16, State 1, Line 260

The statement terminated. The maximum recursion 100 has been exhausted before statement completion.

OPTION maxrecursion

- Example: Give the numbers from 1 to 999

```
WITH numbers(number) AS
(SELECT 1
 UNION all
  SELECT number + 1
  FROM numbers
  WHERE number < 999)

SELECT * FROM numbers
OPTION (maxrecursion 1000);
```

Example application

- Example: Total revenue per month in 2016
Challenge: not all months occur

```
SELECT YEAR(OrderDate)*100 + Month(OrderDate) AS RevenueMonth, SUM(od.UnitPrice * od.Quantity) AS Revenue
FROM Orders o INNER JOIN OrderDetails od ON o.OrderID = od.OrderID
WHERE YEAR(OrderDate) = 2016
GROUP BY YEAR(OrderDate)*100 + Month(OrderDate)
```

	RevenueMonth	Revenue
1	201607	30192,10
2	201608	26609,40
3	201609	27636,00
4	201610	41203,60
5	201611	49704,00
6	201612	50953,40

Example application

- Solution: Generate all months with CTE

```
WITH Months AS  
(SELECT 201601 as RevenueMonth  
 UNION ALL  
 SELECT RevenueMonth + 1  
 FROM Months  
 WHERE RevenueMonth < 201612)  
  
SELECT * FROM Months;
```

	RevenueMonth
1	201601
2	201602
3	201603
4	201604
5	201605
6	201606
7	201607
8	201608
9	201609
10	201610
11	201611
12	201612

Example application

- Solution: ... and combine with LEFT JOIN

```
WITH Months AS
(SELECT 201601 as RevenueMonth
UNION ALL
SELECT RevenueMonth + 1
FROM Months
WHERE RevenueMonth < 201612),

Revenues AS
(SELECT YEAR(OrderDate)*100 + Month(OrderDate) AS RevenueMonth,
SUM(od.UnitPrice * od.Quantity) AS Revenue
FROM Orders o INNER JOIN OrderDetails od ON o.OrderID = od.OrderID
WHERE YEAR(OrderDate) = 2016
GROUP BY YEAR(OrderDate)*100 + Month(OrderDate))

SELECT m.RevenueMonth, ISNULL(r.Revenue, 0) As Revenue
FROM Months m LEFT JOIN Revenues r ON m.RevenueMonth = r.RevenueMonth
```

	RevenueMonth	Revenue
1	201601	0,00
2	201602	0,00
3	201603	0,00
4	201604	0,00
5	201605	0,00
6	201606	0,00
7	201607	30192,10
8	201608	26609,40
9	201609	27636,00
10	201610	41203,60
11	201611	49704,00
12	201612	50953,40

Traversing a hierarchical structure

- Example: Give all employees who report directly or indirectly to Andrew Fuller (**EmployeeID = 2**)
 - Step 1: all employees that report directly to Andrew Fuller
 - Step 2: employees who report to someone from step 1

```
WITH Bosses (boss, emp)
AS
(SELECT ReportsTo, EmployeeID
FROM Employees
WHERE ReportsTo = 2
UNION ALL
SELECT e.ReportsTo, e.EmployeeID
FROM Employees e INNER JOIN Bosses b ON e.ReportsTo = b.emp)

SELECT * FROM Bosses
ORDER BY boss, emp;
```

	boss	emp
1	2	1
2	2	3
3	2	4
4	2	5
5	2	8
6	5	6
7	5	7
8	5	9

Traversing a hierarchical structure

- Draw the organization chart

Now with the
boss included
and with level
and path

	boss	emp	title	level	path
1	NULL	2	Vice President, Sales	1	Vice President, Sales
2	2	1	Sales Representative	2	Vice President, Sales<--Sales Representative
3	2	3	Sales Representative	2	Vice President, Sales<--Sales Representative
4	2	4	Sales Representative	2	Vice President, Sales<--Sales Representative
5	2	5	Sales Manager	2	Vice President, Sales<--Sales Manager
6	2	8	Inside Sales Coordinator	2	Vice President, Sales<--Inside Sales Coordinator
7	5	6	Sales Representative	3	Vice President, Sales<--Sales Manager<--Sales Representative
8	5	7	Sales Representative	3	Vice President, Sales<--Sales Manager<--Sales Representative
9	5	9	Sales Representative	3	Vice President, Sales<--Sales Manager<--Sales Representative

Traversing a hierarchical structure

- Draw the organization chart

```
WITH Bosses (boss, emp, title, level, path)
AS
(SELECT ReportsTo, EmployeeID, Title, 1, convert(varchar(max), Title)
FROM Employees
WHERE ReportsTo IS NULL
UNION ALL
SELECT e.ReportsTo, e.EmployeeID, e.Title, b.level + 1, convert(varchar(max), b.path + '<--' + e.title)
FROM Employees e INNER JOIN Bosses b ON e.ReportsTo = b.emp)

SELECT * FROM Bosses
ORDER BY boss, emp;
```

Exercises

```
-- 1. Give all employees that started working as an employee in the same year as Robert King

-- 2 Make a histogram of the number of orders per customer, so show how many times each number occurs.
-- E.g. in the graph below: 1 customer placed 1 order, 2 customers placed 2 orders, 7 customers placed 3
orders, etc.

/*

nrNumberOfCustomers
11
22
37
46
510
68
77
...

*/
```

Exercises

- 3. Give the customers of the Country in which most customers live
- 4. Give all employees except for the eldest. Solve this first using a subquery and afterwards using a cte
- 5. What is the total number of customers and suppliers?
- 6. Give per title the eldest employee
- 7. Give per title the employee that earns most
- 8. Give the titles for which the eldest employee is also the employee who earns most

Exercises

-- 9. Execute the following script:

```
CREATE TABLE Parts
```

```
(
```

```
  [Super]   CHAR(3) NOT NULL,
```

```
  [Sub]     CHAR(3) NOT NULL,
```

```
  [Amount]  INT NOT NULL,
```

```
  PRIMARY KEY(Super, Sub)
```

```
);
```

```
INSERT INTO Parts VALUES ('01', '02', 10);
```

```
INSERT INTO Parts VALUES ('01', '03', 5);
```

```
INSERT INTO Parts VALUES ('01', '04', 10);
```

```
INSERT INTO Parts VALUES ('02', '05', 25);
```

```
INSERT INTO Parts VALUES ('02', '06', 5);
```

```
INSERT INTO Parts VALUES ('03', '07', 10);
```

```
INSERT INTO Parts VALUES ('06', '08', 15);
```

```
INSERT INTO Parts VALUES ('08', '011', 5);
```

```
INSERT INTO Parts VALUES ('09', '010', 20);
```

```
INSERT INTO Parts VALUES ('010', '011', 25);
```

-- Show all parts that are directly or indirectly part of 02,

-- so all parts of which 02 is composed.

-- Add an extra column with the path as shown:

```
/*
```

```
SUPER SUB PAD
```

```
02 05 02 <-05
```

```
02 06 02 <-06
```

```
06 08 02 <-06 <-08
```

```
08 011 02 <-06 <-08 <-011
```

```
*/
```



Window Functions

Content

- Introduction
- RANGE
- ROWS
- LAG and LEAD

Business case

- Often business managers want to compare current sales to previous sales
- Previous sales can be:
 - **sales during previous month**
 - average sales during last three months
 - last year's sales until current date (year-to-date)
- Window functions offer a solution to these kind of problems in a single, efficient SQL query
- Introduced in SQL: 2003

OVER clause

- Results of a **SELECT** are partitioned
- Numbering, ordering and aggregate functions per partition
- The **OVER** clause creates partitions and ordering
- The partition behaves as a window that shifts over the data
- The OVER clause can be used ..
 - .. with standard aggregate functions (sum, avg, ...)
 - .. with specific window functions (rank, lag,...)

Example: Running total

Make an overview of the UnitsInStock per Category and per Product

```
SELECT CategoryID, ProductID, UnitsInStock
FROM Products
order by CategoryID, ProductID
```

	CategoryID	ProductID	UnitsInStock
1	1	1	39
2	1	2	17
3	1	24	20
4	1	34	111
5	1	35	20
6	1	38	17
7	1	39	69
8	1	43	17
9	1	67	52
10	1	70	15
11	1	75	125
12	1	76	57
13	2	3	13
14	2	4	53
15	2	5	0
16	2	6	120
17	2	8	6
18	2	15	39
19	2	44	27
20	2	61	113
21	2	63	24
22	2	65	76
23	2	66	4

Example

Add extra column with running total of UnitsInStock per Category

- Solution 1 → correlated subquery

```
SELECT CategoryID, ProductID, UnitsInStock,  
(SELECT SUM(UnitsInStock)  
FROM Products  
WHERE CategoryID = p.CategoryID  
and ProductID <= p.ProductID) As TotalUnitsInStockPerCategory  
FROM Products p  
order by CategoryID, ProductID;
```

- Using a correlated subquery is very inefficient as for each line the complete sum is recalculated: $O(n^2)$

	CategoryID	ProductID	UnitsInStock	TotalUnitsInSt...
1	1	1	39	39
2	1	2	17	56
3	1	24	20	76
4	1	34	111	187
5	1	35	20	207
6	1	38	17	224
7	1	39	69	293
8	1	43	17	310
9	1	67	52	362
10	1	70	15	377
11	1	75	125	502
12	1	76	57	559
13	2	3	13	13
14	2	4	53	66
15	2	5	0	66
16	2	6	120	186
17	2	8	6	192
18	2	15	39	231
19	2	44	27	258
20	2	61	113	371
21	2	63	24	395
22	2	65	76	471
23	2	66	4	475

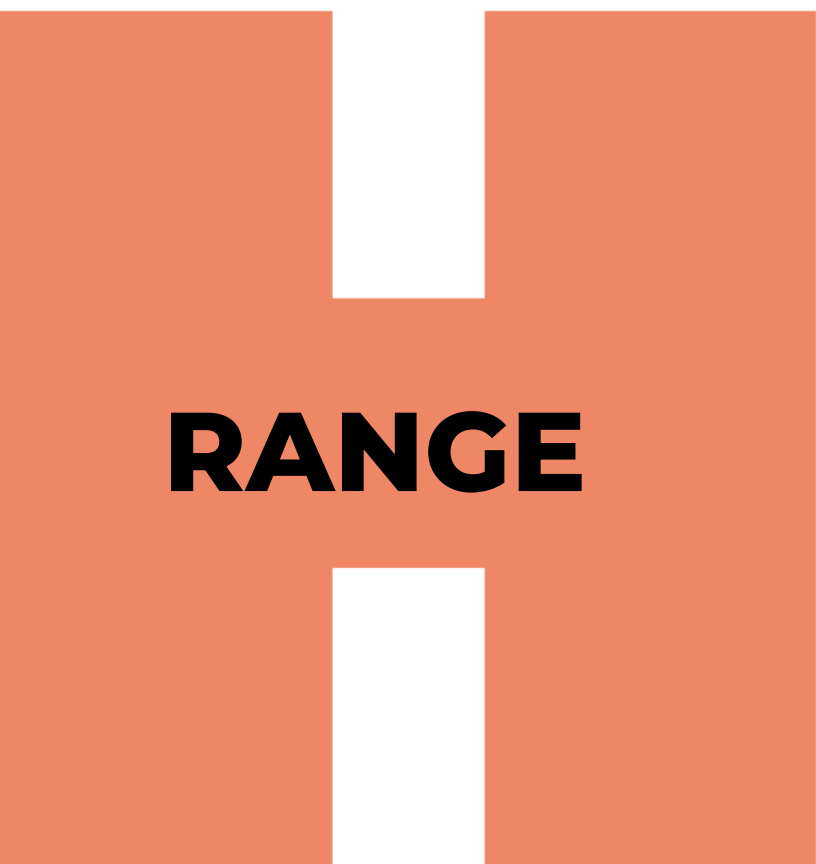
Example

Add extra column with running total of UnitsInStock per Category

- Solution 2 → OVER clause
 - simpler + more efficient
 - The sum is calculated once for each partition

```
SELECT CategoryID, ProductID, UnitsInStock,  
SUM(UnitsInStock) OVER (PARTITION BY CategoryID  
                        ORDER BY ProductID)  
As TotalUnitsInStockPerCategory  
FROM Products
```

	CategoryID	ProductID	UnitsInStock	TotalUnitsInSt...
1	1	1	39	39
2	1	2	17	56
3	1	24	20	76
4	1	34	111	187
5	1	35	20	207
6	1	38	17	224
7	1	39	69	293
8	1	43	17	310
9	1	67	52	362
10	1	70	15	377
11	1	75	125	502
12	1	76	57	559
13	2	3	13	13
14	2	4	53	66
15	2	5	0	66
16	2	6	120	186
17	2	8	6	192
18	2	15	39	231
19	2	44	27	258
20	2	61	113	371
21	2	63	24	395
22	2	65	76	471
23	2	66	4	475

A large, stylized orange letter 'R' that serves as a background for the title. It has a thick stroke and a white rectangular cutout in the center.

RANGE

RANGE clause

- Real meaning of window functions: apply to a window that shifts over the result set
- The previous query works with the default window: start of result set to current row

```
SELECT CategoryID, ProductID, UnitsInStock,  
SUM(UnitsInStock) OVER (PARTITION BY CategoryID ORDER BY ProductID) As TotalUnitsInStockPerCategory  
FROM Products
```

-- The previous query is the shorter version of the following query. Exactly the same resultset!

```
SELECT CategoryID, ProductID, UnitsInStock,  
SUM(UnitsInStock) OVER (PARTITION BY CategoryID ORDER BY ProductID  
RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) As TotalUnitsInStockPerCategory  
FROM Products
```

RANGE clause

- With OVER you have two valid options:
 - `OVER (PARTITION BY <column(s)> ORDER BY <column(s)>)`
 - `OVER (ORDER BY <column(s)>)`
 - (PARTITION is optional, ORDER BY is mandatory)
- With RANGE you have three valid options:
 - `RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW`
 - `RANGE BETWEEN CURRENT ROW AND UNBOUNDED FOLLOWING`
 - `RANGE BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING`

Example

RANGE BETWEEN
UNBOUNDED PRECEDING
AND CURRENT ROW

```
SELECT CategoryID, ProductID, UnitsInStock,  
SUM(UnitsInStock)  
OVER (PARTITION BY CategoryID ORDER BY ProductID  
RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW)  
As TotalUnitsInStockPerCategory  
FROM Products
```

	CategoryID	ProductID	UnitsInStock	TotalUnitsInSt...
1	1	1	39	39
2	1	2	17	56
3	1	24	20	76
4	1	34	111	187
5	1	35	20	207
6	1	38	17	224
7	1	39	69	293
8	1	43	17	310
9	1	67	52	362
10	1	70	15	377
11	1	75	125	502
12	1	76	57	559
13	2	3	13	13
14	2	4	53	66
15	2	5	0	66
16	2	6	120	186
17	2	8	6	192
18	2	15	39	231
19	2	44	27	258
20	2	61	113	371
21	2	63	24	395
22	2	65	76	471
23	2	66	4	475



Example

RANGE BETWEEN
CURRENT ROW AND
UNBOUNDED FOLLOWING

```
SELECT CategoryID, ProductID, UnitsInStock,  
SUM(UnitsInStock)  
OVER (PARTITION BY CategoryID ORDER BY ProductID  
RANGE BETWEEN CURRENT ROW AND UNBOUNDED FOLLOWING)  
As TotalUnitsInStockPerCategory  
FROM Products
```

	CategoryID	ProductID	UnitsInStock	TotalUnitsInStockPerCategory
1	1	1	39	559
2	1	2	17	520
3	1	24	20	503
4	1	34	111	483
5	1	35	20	372
6	1	38	17	352
7	1	39	69	335
8	1	43	17	266
9	1	67	52	249
10	1	70	15	197
11	1	75	125	182
12	1	76	57	57
13	2	3	13	507
14	2	4	53	494
15	2	5	0	441
16	2	6	120	441
17	2	8	6	321
18	2	15	39	315
19	2	44	27	276
20	2	61	113	249
21	2	63	24	136
22	2	65	76	112
23	2	66	4	36

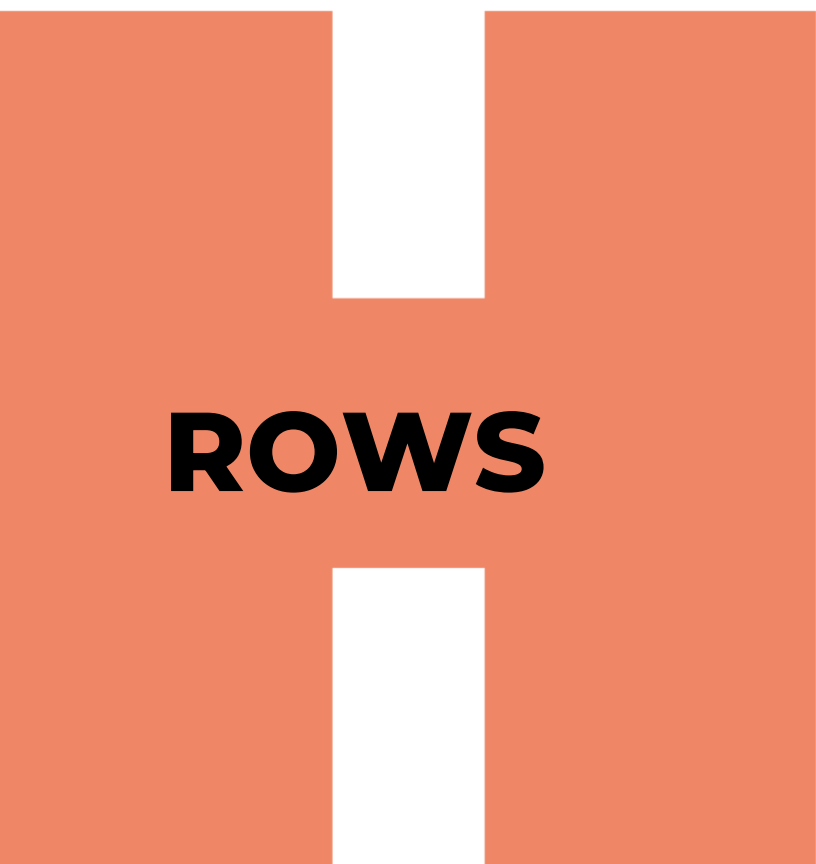
Example

RANGE BETWEEN
UNBOUNDED PRECEDING AND
UNBOUNDED FOLLOWING

```
SELECT CategoryID, ProductID, UnitsInStock,  
SUM(UnitsInStock)  
OVER (PARTITION BY CategoryID ORDER BY ProductID  
RANGE BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING) As  
TotalUnitsInStockPerCategory  
FROM Products
```

	CategoryID	ProductID	UnitsInStock	TotalUnits...
1	1	1	39	559
2	1	2	17	559
3	1	24	20	559
4	1	34	111	559
5	1	35	20	559
6	1	38	17	559
7	1	39	69	559
8	1	43	17	559
9	1	67	52	559
10	1	70	15	559
11	1	75	125	559
12	1	76	57	559
13	2	3	13	507
14	2	4	53	507
15	2	5	0	507
16	2	6	120	507
17	2	8	6	507
18	2	15	39	507
19	2	44	27	507
20	2	61	113	507
21	2	63	24	507
22	2	65	76	507
23	2	66	4	507



A large, stylized orange letter 'R' that serves as a background for the word 'ROWS'.

ROWS

ROWS clause

- With **RANGE**, the current row is compared to all previous/following rows in the partition (based on the ORDER BY predicate)
- With **ROWS**, the current row is compared to n or all previous/following rows in the partition (based on the ORDER BY predicate)
 - ROWS BETWEEN N PRECEDING AND CURRENT ROW
 - ROWS BETWEEN CURRENT ROW AND N FOLLOWING
 - ROWS BETWEEN N PRECEDING AND N FOLLOWING

ROWS clause

- With ROWS you have three valid options:
 - ROWS BETWEEN *N* PRECEDING AND CURRENT ROW
 - ROWS BETWEEN CURRENT ROW AND *N* FOLLOWING
 - ROWS BETWEEN *N* PRECEDING AND *N* FOLLOWING

Example

Overview of salary per employee and average salary of this employee and its 2 predecessors

```
SELECT EmployeeID, FirstName + ' ' + LastName As FullName, Salary,  
AVG(Salary) OVER (ORDER BY Salary DESC ROWS BETWEEN 2 PRECEDING AND CURRENT ROW) As AvgSalary2Preceding  
FROM Employees
```

	EmployeeID	FullName	Salary	AvgSalary2Preceding
1	2	Andrew Fuller	90000.00	90000.000000
2	5	Steven Buchanan	55000.00	72500.000000
3	8	Laura Callahan	51000.00	65333.333333
4	1	Nancy Davolio	48000.00	51333.333333
5	7	Robert King	42000.00	47000.000000
6	4	Margaret Peacock	40000.00	43333.333333
7	9	Anne Dodsworth	40000.00	40666.666666
8	3	Janet Leverling	36000.00	38666.666666
9	6	Michael Suyama	35000.00	37000.000000

Example

Overview of salary per employee and average salary of this employee and its 2 successors

```
SELECT EmployeeID, FirstName + ' ' + LastName As FullName, Salary,  
AVG(Salary) OVER (ORDER BY Salary DESC ROWS BETWEEN CURRENT ROW AND 2 FOLLOWING) As AvgSalary2Following  
FROM Employees
```

	EmployeeID	FullName	Salary	AvgSalary2Following
1	2	Andrew Fuller	90000.00	65333.333333
2	5	Steven Buchanan	55000.00	51333.333333
3	8	Laura Callahan	51000.00	47000.000000
4	1	Nancy Davolio	48000.00	43333.333333
5	7	Robert King	42000.00	40666.666666
6	4	Margaret Peacock	40000.00	38666.666666
7	9	Anne Dodsworth	40000.00	37000.000000
8	3	Janet Leverling	36000.00	35500.000000
9	6	Michael Suyama	35000.00	35000.000000

Example

Overview of salary per employee and average salary of employee, predecessor and successor

```
SELECT EmployeeID, FirstName + ' ' + LastName As FullName, Salary,  
AVG(Salary) (ORDER BY Salary DESC ROWS BETWEEN 1 PRECEDING AND 1 FOLLOWING) As AvgSalary1Preced1Follow  
FROM Employees
```

	EmployeeID	FullName	Salary	AvgSalary1Preceding1Following
1	2	Andrew Fuller	90000.00	72500.000000
2	5	Steven Buchanan	55000.00	65333.333333
3	8	Laura Callahan	51000.00	51333.333333
4	1	Nancy Davolio	48000.00	47000.000000
5	7	Robert King	42000.00	43333.333333
6	4	Margaret Peacock	40000.00	40666.666666
7	9	Anne Dodsworth	40000.00	38666.666666
8	3	Janet Leverling	36000.00	37000.000000
9	6	Michael Suyama	35000.00	35500.000000

Window functions

- **ROW_NUMBER()** returns the sequential number of a row within a partition, starting at 1
 - e.g. 1, 2, 3, 4, 5
- **RANK()** returns the rank of each row within a partition, starting at 1
 - Rank() is the same as row_number(), unless for ties
 - e.g. 1, 2, 2, 4, 5

Window functions

- **DENSE_RANK()** returns the rank of each row within a partition, starting at 1, without gaps
 - e.g. 1, 2, 2, 3, 4
- **PCT_RANK()** shows the ranking of each row within a partition on a scale from 0 to 1

Example

ROW_NUMBER / RANK / DENSE_RANK / PERCENT_RANK
for each employee based on their salary

```
SELECT EmployeeID, FirstName + ' ' + LastName As 'Full Name', Title, Salary,  
ROW_NUMBER() OVER (ORDER BY Salary DESC) As 'ROW_NUMBER',  
RANK() OVER (ORDER BY Salary DESC) As 'RANK',  
DENSE_RANK() OVER (ORDER BY Salary DESC) As 'DENSE_RANK',  
PERCENT_RANK() OVER (ORDER BY Salary DESC) As 'PERCENT_RANK'  
FROM Employees
```

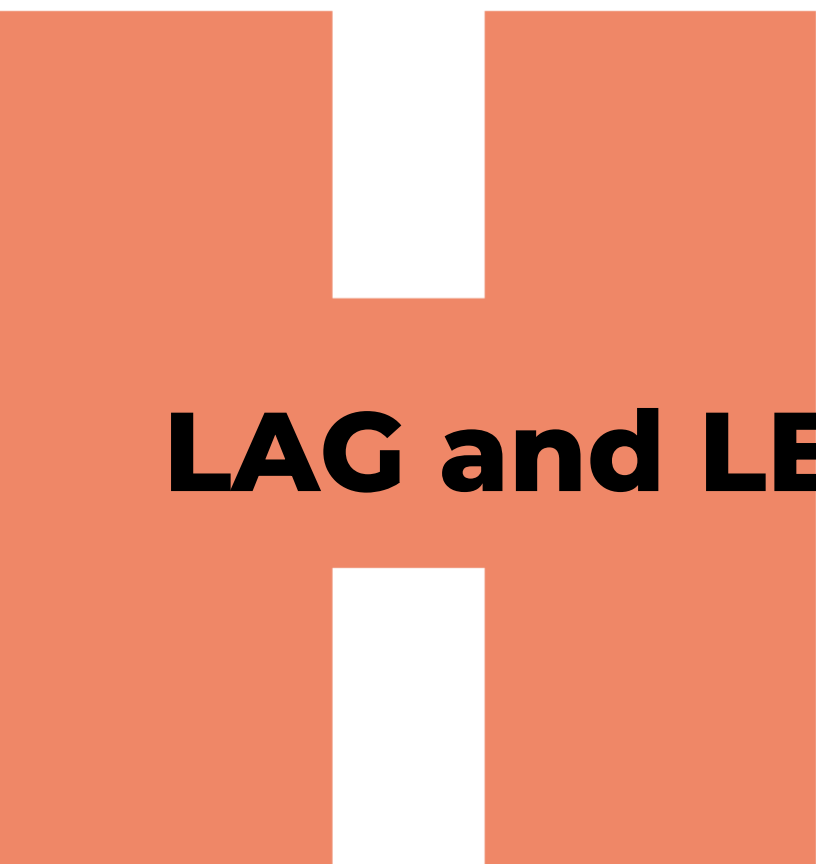
	EmployeeID	Full Name	Title	Salary	ROW_NUMBER	RANK	DENSE_RANK	PERCENT_RANK
1	2	Andrew Fuller	Vice President, Sales	90000.00	1	1	1	0
2	5	Steven Buchanan	Sales Manager	55000.00	2	2	2	0,125
3	8	Laura Callahan	Inside Sales Coordinator	51000.00	3	3	3	0,25
4	1	Nancy Davolio	Sales Representative	48000.00	4	4	4	0,375
5	7	Robert King	Sales Representative	42000.00	5	5	5	0,5
6	4	Margaret Peacock	Sales Representative	40000.00	6	6	6	0,625
7	9	Anne Dodsworth	Sales Representative	40000.00	7	6	6	0,625
8	3	Janet Leverling	Sales Representative	36000.00	8	8	7	0,875
9	6	Michael Suyama	Sales Representative	35000.00	9	9	8	1

Example

ROW_NUMBER / RANK / DENSE_RANK / PERCENT_RANK
per title for each employee based on their salary

```
SELECT EmployeeID, FirstName + ' ' + LastName As 'Full Name', Title, Salary,  
ROW_NUMBER() OVER (PARTITION BY Title ORDER BY Salary DESC) As 'ROW_NUMBER',  
RANK() OVER (PARTITION BY Title ORDER BY Salary DESC) AS 'RANK',  
DENSE_RANK() OVER (PARTITION BY Title ORDER BY Salary DESC) AS 'DENSE_RANK',  
PERCENT_RANK() OVER (PARTITION BY Title ORDER BY Salary DESC) AS 'PERCENT_RANK'  
FROM Employees
```

	EmployeeID	Full Name	Title	Salary	ROW_NUMBER	RANK	DENSE_RANK	PERCENT_RANK
1	8	Laura Callahan	Inside Sales Coordinator	51000.00	1	1	1	0
2	5	Steven Buchanan	Sales Manager	55000.00	1	1	1	0
3	1	Nancy Davolio	Sales Representative	48000.00	1	1	1	0
4	7	Robert King	Sales Representative	42000.00	2	2	2	0.2
5	4	Margaret Peacock	Sales Representative	40000.00	3	3	3	0.4
6	9	Anne Dodsworth	Sales Representative	40000.00	4	3	3	0.4
7	3	Janet Leverling	Sales Representative	36000.00	5	5	4	0.8
8	6	Michael Suyama	Sales Representative	35000.00	6	6	5	1
9	2	Andrew Fuller	Vice President, Sales	90000.00	1	1	1	0



LAG and LEAD

LAG and LEAD

- LAG refers to the previous line. This is short for LAG(..., 1)
- LAG(..., 2) refers to the line before the previous line, ...
- LEAD refers to the next line. This is short for LEAD(..., 1)
- LEAD(..., 2) refers to the line after the next line, ...

Example: LAG

Calculate for each employee the percentage difference between this employee and the employee preceding him

```
SELECT EmployeeID, FirstName + ' ' + LastName, Salary,  
FORMAT((Salary - LAG(Salary) OVER (ORDER BY Salary DESC)) / Salary, 'P') As EarnsLessThanPreceding  
FROM Employees
```

	EmployeeID	(No column name)	Salary	EarnsLessThanPreceding
1	2	Andrew Fuller	90000.00	NULL
2	5	Steven Buchanan	55000.00	-63.64%
3	8	Laura Callahan	51000.00	-7.84%
4	1	Nancy Davolio	48000.00	-6.25%
5	7	Robert King	42000.00	-14.29%
6	4	Margaret Peacock	40000.00	-5.00%
7	9	Anne Dodsworth	40000.00	0.00%
8	3	Janet Leverling	36000.00	-11.11%
9	6	Michael Suyama	35000.00	-2.86%

Example: LEAD

Calculate for each employee the percentage difference between this employee and the employee following him

```
SELECT EmployeeID, FirstName + ' ' + LastName, Salary,  
FORMAT((Salary - LEAD(Salary) OVER (ORDER BY Salary DESC)) / Salary, 'P') As EarnsMoreThanFollowing  
FROM Employees
```

	EmployeeID	(No column name)	Salary	EarnsMoreThanFollowing
1	2	Andrew Fuller	90000.00	38.89%
2	5	Steven Buchanan	55000.00	7.27%
3	8	Laura Callahan	51000.00	5.88%
4	1	Nancy Davolio	48000.00	12.50%
5	7	Robert King	42000.00	4.76%
6	4	Margaret Peacock	40000.00	0.00%
7	9	Anne Dodsworth	40000.00	10.00%
8	3	Janet Leverling	36000.00	2.78%
9	6	Michael Suyama	35000.00	NULL

Exercises

Exercises

```
-- Exercise 1
-- Create the following overview in which each customer gets a sequential number.
-- The companynames are sorted alphabetically
-- The number is reset when the country changes
/*
country      rownum      CompanyName
Argentina    1          Cactus Comidas para llevar
Argentina    2          Océano Atlántico Ltda.
Argentina    3          Rancho grande
Austria      1          Ernst Handel
Austria      2          Piccolo und mehr
Belgium      1          Maison Dewey
Belgium      2          Suprêmes délices
Brazil       1          Comércio Mineiro
Brazil       2          Família Arquibaldo
Brazil       3          Gourmet Lanchonetes
Brazil       4          Hanari Carnes
...
*/
```

Exercises

```
-- Exercise 2
-- First create an overview that shows for each productid the amount sold per year

-- Now create an overview that shows for each productid the amount sold per year and for the previous year.
/*
1      2016  125   NULL
1      2017  304   125
1      2018  399   304
2      2016  226   NULL
2      2017  435   226
2      2018  396   435
3      2016   30   NULL
3      2017  190   30
3      2018  108   190
...
*/
```

Exercises

```
-- Use a CTE and the previous SQL Query to calculate the year over year performance for each productid.  
-- If the amountPreviousYear is NULL, then the year over year performance becomes N/A.
```

```
/*  
1      2016   125    NULL   N/A  
1      2017   304    125    143.20%  
1      2018   399    304    31.25%  
2      2016   226    NULL   N/A  
2      2017   435    226    92.48%  
2      2018   396    435    -8.97%  
3      2016   30     NULL   N/A  
3      2017   190    30     533.33%  
3      2018   108    190    -43.16%  
...  
*/
```

Exercises

```
-- Exercise 3
-- Which is the most popular shipper
-- Step 1: Use a CTE and add DENSE_RANK
-- Step 2: FILTER on DENSE_RANK = 1

-- ShipperID CompanyName ShipVia      NumberOfOrders  DENSE_RANK
-- 2 United Package 2 326 1
```

Exercises

```
-- Exercise 4
-- Which is the TOP 3 of countries in which most customers live?
-- Step 1: Use a CTE and add DENSE_RANK
-- Step 2: FILTER on DENSE_RANK
```

--Country	NumberOfCustomers	DENSE_RANK
--USA 13	1	
--France 11	2	
--Germany 11	2	
--Brazil 9	3	

Exercises

```
-- Exercise 5
-- Imagine there is a bonussystem for all the employees: the best employee gets 10 000EUR bonus, the second
-- one 5000 EUR, the third one 2500 EUR, ...
-- Let's calculate the bonus for each employee, based on the revenue per year per employee
-- First create an overview of the revenue (unitprice * quantity) per year per employeeid
/*
1      2016   38789,00
1      2017   97533,58
1      2018   65821,13
2      2016   22834,70
2      2017   74958,60
2      2018   79955,96
3      2016   19231,80
3      2017   111788,61
3      2018   82030,89
4      2016   53114,80
4      2017   139477,70
4      2018   57594,95
...
*/
```

Exercises

```
-- Now add a ranking per year per employeeid
/*
4      2016   53114,80      1
1      2016   38789,00      2
8      2016   23161,40      3
2      2016   22834,70      4
5      2016   21965,20      5
3      2016   19231,80      6
7      2016   18104,80      7
6      2016   17731,10      8
9      2016   11365,70      9
...
*/
```

Exercises

-- Imagine there is a bonussystem for all the employees: the best employee gets 10 000EUR bonus, the second one 5000 EUR, the third one 2500 EUR, ...

```
/*  
4      2016  53114,80      10000  
1      2016  38789,00      5000  
8      2016  23161,40      3333  
2      2016  22834,70      2500  
5      2016  21965,20      2000  
3      2016  19231,80      1666  
7      2016  18104,80      1428  
6      2016  17731,10      1250  
9      2016  11365,70      1111  
...  
*/
```

Exercises

```
-- Exercise 6: Calculate for each month the percentage difference between the revenue for this month and the
previous month
/*
2016  7      30192,10      NULL  NULL
2016  8      26609,40      30192,10    -11.86%
2016  9      27636,00      26609,40     3.85%
2016 10      41203,60      27636,00    49.09%
2016 11      49704,00      41203,60    20.63%
2016 12      50953,40      49704,00     2.51%
2017  1      66692,80      50953,40    30.88%
2017  2      41207,20      66692,80   -38.21%
...
*/
-- Step 1: calculate the revenue per year and per month
-- Step 2: Add an extra column for each row with the revenue of the previous month
-- Step 3: Calculate the percentage difference between this month and the previous month
```

Table Operators:

- (CROSS) APPLY**
- PIVOT**
- UNPIVOT**

APPLY

- The APPLY operator operates on two input tables, the second of which can be a table expression;
- We refer to them as the “left” and “right” tables. The right table is usually a derived table or an inlineTVF.
- The CROSS APPLY operator implements one logical query processing phase—it applies the right table expression to each row from the left table and produces a result table with the unified result sets.
- So far it might sound like the CROSS APPLY operator is very similar to a cross join, and in a sense that’s true. For example, the following two queries return the same result sets.

CROSS APPLY

```
SELECT c.courierid,e.employeeid  
from courier c cross join employee  
e;
```

```
SELECT c.courierid,e.employeeid  
from courier c cross apply employee  
e;
```

CROSS APPLY

- However, with the *CROSS APPLY* operator, the right table expression can represent a different set of rows per each row from the left table, unlike in a join.
- You can achieve this when you use a derived table in the right side, and in the derived table query refer to attributes from the left side.
- Alternatively, when you use an inline TVF, you can pass attributes from the left side as input arguments.
- For example, the following code uses the *CROSS APPLY* operator to return the three most recent orders for each customer.

Cross apply

Database xtreme: all orders per customer.

```
select c.CustomerID, c.CustomerName, o.OrderID, o.OrderDate
from customer c join orders o
on c.CustomerID=o.CustomerID
order by c.CustomerID,o.OrderDate desc;
```

Date of latest order per customer.

```
select c.CustomerID,c.CustomerName,max(o.OrderDate)
from customer c join orders o
on c.CustomerID=o.CustomerID
group by c.CustomerID,c.CustomerName
order by c.CustomerID desc;
```

Cross apply

Latest order(s) per customer.

```
with recent(custnr,custname,ordate) as
(select
c.CustomerID,c.CustomerName,max(o.OrderDate)
from customer c join orders o
on c.CustomerID=o.CustomerID
group by c.CustomerID,c.CustomerName)
select custnr,custname,ordate,o.orderid
from recent join orders o
on recent.custnr=o.customerid
where recent.ordate=o.orderdate
```

Cross apply

3 latest orders per customer

```
select
c.customerid,c.customername,o.orderdate,o.orderid
from customer c
cross apply
(select top(3) orderid,orderdate
from orders where customerid = c.customerid order
by orderdate desc,orderid) as o
order by customerid;
```

Cross apply

- You can think of the table expression *o* as a correlated table subquery.
- In terms of logical query processing, the right table expression (a derived table, in this case) is applied to each row from the *Customers* table.
- Notice the reference to the attribute *c.customerid* from the left table in the derived table's query filter.
- The derived table returns the three most recent orders for the customer from the current left row.
- Because the derived table is applied to each row from the left side, the *CROSS APPLY* operator returns the three most recent orders for each customer.

Outer apply

3 latest orders per customer, customers without any orders should also be on the list (e.g. customerid = 78)

```
select c.customerid,c.customername,o.orderdate,o.orderid
from customer c
outer apply
(select top(3) orderid,orderdate
from orders o where customerid = c.customerid order by
orderdate desc,orderid) o;
```

As with outer-join missing data are shown as NULL.

231	77	Warsaw Sports, Inc.	2002-03-21 00:00:00	2842
232	78	Greenlane Bicycles	NULL	NULL
233	79	Helsinki Bicycle	2001-05-07 00:00:00	3001

Cross apply with inline TVF

```
IF OBJECT_ID('dbo.TopOrders') IS NOT NULL
    DROP FUNCTION dbo.TopOrders;

GO

CREATE FUNCTION dbo.TopOrders (@custid AS INT, @n AS INT)
    RETURNS TABLE
AS
    RETURN
        SELECT TOP (@n) orderid, employeeid, orderdate,
            requireddate
        FROM Orders
        WHERE customerid = @custid
        ORDER BY orderdate DESC, orderid DESC;

GO
```

Cross apply with inline TVF

```
select c.customerid, c.customername,  
o.orderdate,o.orderid  
from customer c  
cross apply  
dbo.toporders(c.customerid,3) as o  
order by customerid;
```

- This code is much more readable and easier to maintain.
- In terms of physical processing, nothing really changed because SQL Server will in any case end up querying the underlying objects directly.

Pivot with case statement

- How many products of each class do we buy per supplier country ?

```
select p.productclassid,  
sum(case when s.country='USA' then 1 else 0 end) as 'USA',  
sum(case when s.country='Canada' then 1 else 0 end) as 'Canada',  
sum(case when s.country='Japan' then 1 else 0 end) as 'Japan',  
sum(case when s.country='UK' then 1 else 0 end) as 'UK',  
count(productid) TOTAAL  
from product p join supplier s  
on p.supplierid = s.supplierid  
group by p.productclassid;
```

	class	usa	japan	canada	uk	TOTAL
1	1	27	11	20	3	61
2	2	0	0	56	0	56

Pivot with pivot statement

```
select productclassid,USA,Japan,Canada,UK
from
(
select p.productclassid,s.country
from product p join supplier s
on p.SupplierID=s.supplierid
) p
pivot
(
count(country) for country in
(USA,Japan,Canada,UK)
) as pvt;
```

productclassid	USA	Japan	Canada	UK
1	27	11	20	3
2	0	0	55	0

Pivot with pivot statement

```
-- with totals
select
productclassid,USA,Japan,Canada,UK,USA+Japan+Canada+UK
Total
from
(
select p.productclassid,s.country
from product p join supplier s
on p.SupplierID=s.supplierid
) as p
pivot
(
count(country) for country in (USA,Japan,Canada,UK)
) as pvt
```

	class	usa	japan	canada	uk	TOTAL
1	1	27	11	20	3	61
2	2	0	0	56	0	56

Pivot with pivot statement

- The PIVOT operator operates in the context of the FROM clause of a query like other table operators (for example, JOIN).
- It operates on a source table or table expression, pivots the data, and returns a result table.
- In the parentheses of the *PIVOT* operator, you specify an aggregate function (COUNT, in this example), aggregation element (*country*), spreading element (*country*), and the list of target column names (USA,Japan,Canada,UK).
- Following the parentheses of the *PIVOT* operator, you specify an alias for the result table.

Pivot with pivot statement

- As another example of a pivoting request, suppose that instead of returning product classes on rows and countries on columns, you want it the other way around: the grouping element is *country*, the spreading element is *productclassid*, and the aggregation element and aggregate function remain `count(country)`.
- After you learn the template for pivoting you fit those elements in the right places.
- The following solution query uses the native PIVOT operator to achieve the result.

Pivot with pivot statement

```
select country,[1],[2]
from
(
  select country,productclassid
  from product p join supplier s
  on p.SupplierID=s.supplierid
) as p
pivot
(
  count(productclassid) for productclassid
  in ([1],[2])
) as pvt ;
```

Pivot with pivot statement

- The productclassIDs 1 and 2 are values in the productclassid column in the source table, but in terms of the result, these values become target column names.
- Therefore, in the PIVOT IN clause, you must refer to them as identifiers.
- When identifiers are irregular (for example, when they start with a digit), you need to delimit them—hence the use of square brackets.

Exercise

- Calculate for each employee and per country of the customer the total order amount of the orders handled by that employee in the countries Belgium, France and Germany.
 - Rows = lastname-firstname
 - Columns = country
 - Cells = total order amount

Unpivot

- Unpivoting is a technique to rotate data from a state of columns to a state of rows.
- Usually it involves querying a pivoted state of the data, producing from each source row multiple result rows, each with a different source column value.
- In other words, each source row of the pivoted table becomes potentially many rows, one row for each of the specified source column values.

Unpivot

```
-- create pivot data
IF OBJECT_ID('dbo.ClassCountry', 'U') IS NOT
NULL DROP TABLE dbo.ClassCountry;
CREATE TABLE dbo.ClassCountry
( class int NOT NULL PRIMARY KEY,
  usa int NULL, japan int NULL, canada int
NULL, uk int NULL, total int NULL );

insert into ClassCountry
select
productclassid,USA,Japan,Canada,UK,USA+Japan
+Canada+UK Total
from
(
    select p.productclassid,s.country
    from product p join supplier s
    on p.SupplierID=s.supplierid
    where productclassid is not null
) as p
pivot
(
count(country) for country in
(USA,Japan,Canada,UK)
) as pvt ;
```

Unpivot

```
select class,country,cntproducts
from classcountry
unpivot (cntproducts for country in
(USA,Japan,Canada,UK)) as u;
```

- Like the PIVOT operator, UNPIVOT was also implemented as a table operator in the context of the FROM clause.
- It operates on a source table or table expression (classcountry in this case).
- Within the parentheses of the UNPIVOT operator, you specify the name you want to assign to the column that will hold the source column values (cntproducts here), the name you want to assign to the column that will hold the source column names (country), and the list of source column names (USA,Japan,Canada and UK).
- Following the parentheses, you provide an alias to the table resulting from the table operator.